

Comunicatia seriala a datelor prin UART (Universal Asynchronous Receiver/Transmitter)

1 Scopul lucrarii

Studentul ar trebui ca, la sfarsitul sedintei de laborator, sa aiba cunostinte de baza despre comunicatia seriala a datelor prin UART (Universal Asynchronous Receiver/Transmitter) si sa utilizeze practic (in programe asm) registrele speciale ale acestui sistem.

2 Introducere

In acest capitol vor fi prezentate:

- 2.1 Comunicatii seriale
- 2.2 UART1
- 2.3 Timer1
- 2.4 PuTTY

2.1 Comunicatii seriale

2.1.1 Parametri

Comunicatia seriala presupune transformarea informatiei stocata sub forma de octeti intr-un sir de biti consecutivi format din biti de date si biti de control, transmiterea acestui sir de biti printr-un mediu fizic si reconstituirea, la receptie a informatiei initiale.

In general se transmit simboluri, de aceea frecventa de transmisie este denumita baud rate (frecventa simbolurilor), insa, in cazul particular in care simbolurile sunt chiar biti, baud rate-ul este de fapt rata de bit.

Transmisia seriala presupune si introducerea unor biti de control, astfel incat la receptie sa se poata reconstitui informatia originala. Astfel, bitii transmisi intr-o comunicatie seriala pot fi:

- biti de start
- biti de date
- biti de stop
- biti de paritate

2.1.2 UART

UART este un dispozitiv care translateaza datele intre interfetele seriale si paralele. Folosit in comunicatia seriala, UART-ul converteste datele sub forma de octeti in si din siruri de biti (biti de date delimitati de biti de start si stop).

Standardul UART defineste structura sirului de biti astfel:

- un bit de start (nivel logic 0)
- cinci pana la opt biti de date
- un bit de optional de paritate (suma bitilor de date)
- unul, unu si jumatate sau doi biti de stop (nivel logic 1)

2.1.3 RS-232

In general, UART-ul nu realizeaza si transmisia efectiva a datelor pe mediul fizic. De obicei o interfata seriala este utilizata pentru transformarea nivelelor logice de tensiune cu care lucreaza UART-ul in nivelele semnalului extern (semnalul care circula efectiv prin mediul fizic).

O astfel de interfata este definita de standardul EIA RS-232, acest tip de interfata gasindu-se si pe placile de dezvoltare din laborator.

2.2 UART1

Microcontrolerul C8051F040 dispune de doua sisteme de transmisie-receptie asincrone, UART0 si UART1. In aceasta lucrare, vom detalia modul de functionare al UART1.

UART1 este un port serial, full-duplex, asincron, care poate functiona in modurile 8-bit UART si 9-bit UART definite in standardul 8051 UART.

UART1 are asociate 2 registre speciale: Serial Control Register (SCON1) și Serial Data Buffer (SBUF1). Registrul SBUF1 este de fapt un buffer care interfateaza utilizatorul cu registrele fizice de transmisie si receptie ale UART-ului. O operatie de scriere a acestui registru special determina inceputul unei transmisii, intrucat datele sunt trimise catre registrul shift de transmisie, in timp ce o operatie de citire a acestui registru acceseaza latch-ul de receptie al UART-ului.

Dacă intreruperile asociate UART1 sunt activate, o intrerupere va fi generata de fiecare data cand o transmisie este completa (bitul TI1 din registrul SCON1 este setat), sau un byte de date a fost receptionat (bitul RI1 din registrul SCON1 este setat). Flag-urile de intrerupere (TI1 si RI1) nu sunt resetate de hardware, ci trebuie resetate din software, dupa ce s-a stabilit cauza intreruperii (transmisie completa / receptie). Pentru mai multe informatii despre definitia SCON1 si definitia SBUF1 consultati documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 284, pag. 285).

2.2.1 Modurile de functionare ale UART1

UART1 poate functiona in unul din modurile 8-bit UART, respectiv 9-bit UART, in functie de valoarea logica a bitului S1MODE al registrului special SCON1.

2.2.1.1 Modul 8-bit

In acest mod de operare un octet de informatie este transformat intr-un sir de 10 biti astfel:

- primul bit este un bit de start
- urmatorii 8 biti sunt bitii de informatie, ordonati incepand cu cel mai putin semnificativ
- ultimul bit este un bit de stop

Datele se transmit pe pinul TX1 si se receptioneaza pe pinul RX1. La receptie, cei 8 biti de date sunt stocati in registrul SBUF1, iar al noualea bit (cel de stop in cazul acestui mod de operare) este stocat in bitul RB81 al registrului SCON1.

Transmisia incepe cand un octet este scris in registrul SBUF1. Flag-ul TI1 este automat setat la sfarsitul fiecarei transmisii (in momentul transmiterii bitului de stop). Receptia datelor poate incepe oricand dupa momentul setarii bitului Receive Enable bit (bitul 4 al registrului SCON1). Dupa ce s-a receptionat bitul de stop octetul receptionat este incarcat din registrul fizic de receptie in registrul SBUF1 numai daca bitul RI1 are valoarea logica 0. Toate datele receptionate cat timp acest bit este 1 sunt ignorate pentru ca microcontrolerul considera ca octetul receptionat in prealabil nu a fost inca prelucrat.

2.2.1.2 Modul 9-bit

In acest mod de operare un octet de informatie este transformat intr-un sir de 11 biti astfel:

- primul bit este un bit de start
- urmatorii 8 biti sunt bitii de informatie, ordonati incepand cu cel mai putin semnificativ
- urmatorul bit este un bit de control programabil
- ultimul bit este un bit de stop

Tipul bitului al noualea este determinat de valoarea bitului TB81 (bitul 3 al registrului SCON1). Acest bit poate fi setat ca bit de paritate sau ca bit de semnalizare in comunicatia multiprocesor. La receptie, acest al noualea bit este stocat in bitul RB81 al registrului SCON1. Transmisia si receptia se realizeaza in acelasi fel ca in modul 8-bit.

2.2.1.3 Generarea baud-rate

Baud rate-ul pentru UART1 este generat de Timerul 1 configurat in Modul 2 (8-bit auto-reload). Mai multe informatii despre Timerul 1 se gasesc in capitolul 2.3.1.

Generarea baud rate-ului de transmisie:

- Se folosesc depasirile Timerului 1 (TL1).
- Frecventa de transmisie a simbolurilor este jumata din frecventa depasirilor Timerului 1 (TL1).
- Reincarcarea TL1 (cu valoarea din TH1) este determinata de o depasire.

Generarea baud rate-ului de receptie:

- Se folosesc depasirile unei copii a timerului 1 (RX Timer).
- Frecventa de receptie a simbolurilor este jumata din frecventa depasirilor acestei copii.
- Reincarcarea RX Timer (cu valoarea din TH1) este determinata de o depasire sau de detectia unui bit de start – acest lucru permite ca receptia sa inceapa la orice moment de timp, indiferent de starea Timerului 1.

Generarea baud rate-ului este prezentata in Fig. 2.2.1.3.

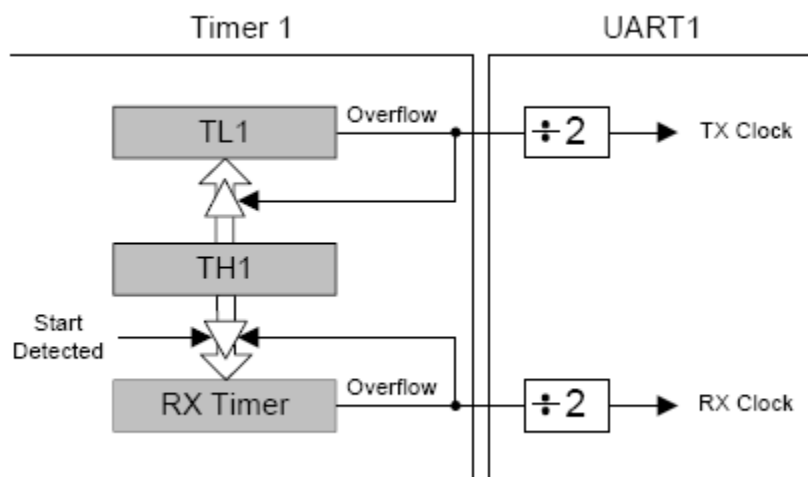


Fig. 2.2.1.3. Generarea baud rate-ului pentru UART1

Valoarea de reload a timerului 1 (TH1) trebuie setata astfel incat depasirile sa apara la dublul frecventei simbolurilor (baud rate) dorite. Timerul 1 poate fi sincronizat cu una din urmatoarele surse: SYSCLK, SYSCLK/4, SYSCLK/12, SYSCLK/48, sau cu un frecventa unui oscilator extern divizata cu 8. Frecventa simbolurilor poate fi determinata din ecuatia de mai jos, unde T1CLK este frecventa ceasului cu care este sincronizat Timer1, iar TH1 este valoarea de reload a Timerului 1.

$$\text{UartBaudRate} = 0.5 * \text{T1CLK} / (256 - \text{TH1})$$

Observatie: SYSCLK este frecventa de ceas a sistemului.

Tabelele Table 22.1 pana la Table 22.6 din documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 286 - 288) stabilesc valorile T1CLK si TH1 pentru diferite baud rate-uri.

2.3 Timer-ele

Microcontrolerul C8051F040 dispune de cinci timere pe 16 biti: Timer 0, Timer 1 (specifice standardului 8051) si Timer 2, Timer 3, Timer 4 (specifice acestui microcontroler si utilizate de obicei in corespondenta cu convertoarele analog-digitale).

2.3.1 Timer 1

- este un timer/counter implementat cu un registru pe 16 biti accesibili pe octeti astfel: un octet inferior (TL1) si un octet superior (TH1).
- poate fi sincronizat cu una din urmatoarele surse de ceas:
 - ceasul de sistem
 - ceasul de sistem cu frecventa divizata prin 4, 12 sau 48
 - un oscilator extern cu frecventa divizata prin 8in functie de configuratia bitilor T1M, SCA1 si SCA0 din registrul CKCON. Pentru mai multe informatii despre definitia CKCON consultati documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 295).
- poate functiona ca numarator, contorizand fronturile cazatoare ale unui semnal extern aplicat pe un pin al unui port de uz general. Nu este obligatoriu ca semnalul sa fie periodic, insa trebuie ca frecventa lui sa nu depaseasca un sfert din frecventa ceasului intern, iar palierele semnalului sa dureze cel putin doua perioade al ceasului intern.
- registrul TCON este folosit pentru activarea timerului, cat si pentru a verifica statusul acestuia. Pentru mai multe informatii despre definitia TCON consultati documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 293).
- bitii T1M1 – T1M0 din registrul TMOD stabilesc functionarea timerului intr-unul din urmatoarele trei moduri:
 - modul 0 – 13-bit counter/timer
 - modul 1 – 16-bit counter/timer
 - modul 2 – 8-bit auto-reload counter/timerPentru mai multe informatii despre definitia TMOD consultati documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 294).

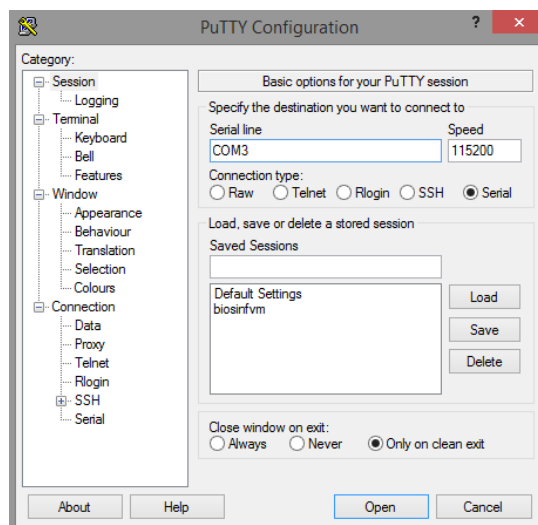
2.3.1.1 Functionarea Timer 1 in modul 2 (8-bit auto-reload)

În acest mod de funcționare valoarea stocată în TL1 se incrementează la fiecare tact al sursei de ceas cu care este sincronizat timerul, iar TH1 reține valoarea de reload. În momentul tranziției de la valoarea 0xFF la 0x00 (depășire), registrul TL1 este reîncărcat cu valoarea stocată în TH1, iar bitul TF1 (bitul 7 al registrului TCON) este setat. Dacă întreruperile timerului 1 sunt activate, atunci o întrerupere este generată în momentul setării bitului TF1. Valoarea stocată în TH1 nu este modificată pe parcursul funcționării.

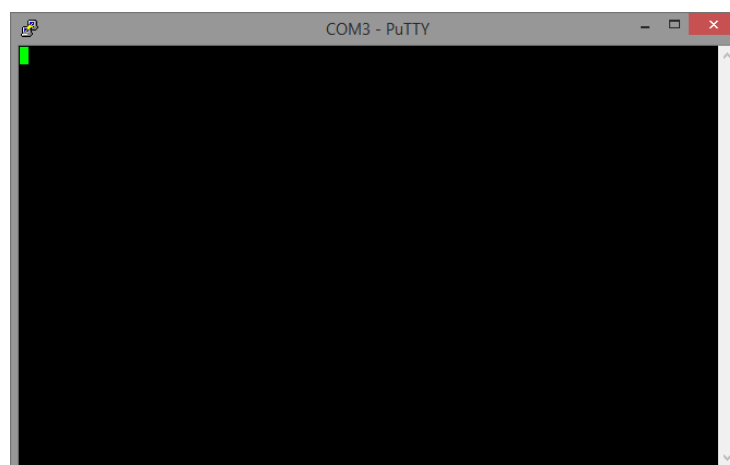
2.4 PuTTY

PuTTY este un emulator de terminal, ce ajută la comunicarea cu alte dispozitive, folosind diverse protocoale (SSH, Serial, etc.). Pentru a stabili o conexiune de tip serial între microcontroller și computer, se urmăresc pașii:

- Se deschide PuTTY, folosind scurtătura de pe Desktop: 
- Se configurează astfel: tipul conexiunii (Serial), valoarea baud rate-ului (Speed = 115200) și numele portului pe care se realizează comunicarea (Serial line = COM1 pentru calculatoare sau Serial line = COM3 pentru laptopuri):



- Dupa realizarea configurării, se apasă Open și se va deschide un terminal unde va scrie (pentru transmisie), dar se va putea vizualiza ceea ce se recepționează:



Problema

2.5 Formularea problemei

Se cere sa se implementeze un sistem de comunicatie seriala intre calculator si microcontrolerul C8051F040. Se va utiliza programul PuTTY si un cablu serial pentru a transmite microcontrolerului un caracter ASCII intre A si Z, iar acesta va raspunde transmitand inapoi caractere ASCII incepand cu caracterul receptionat si incheind cu caracterul Z.

Observatie: American Standard Code for Information Interchange (prescurtat ASCII) este unul din cele mai vechi standarde de codare a caracterelor. Tabela ASCII reprezinta o corespondenta biunivoca intre numerele 0-255 (numere ce “incap” in 8 biti) si anumite simboluri grafice (inclusiv caracterele 0-9, a-z, A-Z). Anumite coduri ASCII reprezinta caractere de control si nu simboluri grafice (de exemplu: end of text, backspace, line feed, form feed, etc). Tabelul ASCII este prezentat in Anexa 5.2.

2.6 Solutie posibila

2.6.1 Descrierea modului hardware

Vom prezenta in continuare o implementare a unui sistem avand la baza placa de dezvoltare cu microcontrolerul C8051F040. Placa de dezvoltare (portul serial etichetat J5) este conectata cu calculatorul (portul COM1) prin intermediul unui cablu serial.

2.6.2 Descrierea algoritmului

Observatie: programul respecta structura generala prezentata in Lucrarea nr.1, capitolul 3.2.2.

In solutia propusa in acest laborator programul va avea mai multe rutine: `main`, `init`, `sendByte` si `receiveByte`.

Algoritmul consta intr-o bucla care se autoapeleaza la infinit etichetata `start`. In interiorul acestei bucle se fac cateva operatii:

- se apeleaza subrutina `receiveByte` care receptioneaza un octet si il stocheaza in acumulator,
- se compara pe rand (prin scadere) valoarea acumulatorului cu codurile ASCII ale caracterelor „A”, respectiv „Z” si, in cazul in care caracterul receptionat este in afara acestei game se face un salt la eticheta `error`. Observatii:
 - valoarea receptionata este in prealabil salvata in registrul B, intrucat valoarea din registrul A este modificata in urma acestor operatii,
 - saltul la eticheta `error` se face daca fanionul CY (Carry Flag) din registrul PSW este setat. Acest fanion este setat automat daca la scadere este nevoie de „imprumut”, adica daca cel de-al doilea operand este mai mare.
- in continuare se stocheaza codul ASCII al caracterului receptionat in registrul R0, si se initializeaza registrul R1 cu numarul de caractere ce se vor transmite prin UART1. R0 este utilizat pentru a stoca codurile ASCII ale caracterelor ce va fi afisate in PuTTY, iar R1 are rolul de contor in bucla de afisare;
- se intra in bucla de transmisie etichetata `loop`, care are scopul de a transmite caracterele corespunzatoare (incepand cu caracterul receptionat si incheind cu caracterul Z) astfel:
 - se apeleaza subrutina `sendByte` care transmite codul stocat in R0,
 - se incrementeaza valoarea lui R0 (trecand astfel la urmatorul caracter),
 - se decrementeaza contorul buclei de afisare, iar daca acesta nu s-a anulat se sare la o noua iteratie de afisare.
- in cazul in care s-a ajuns la eticheta `error` se folosesc aceleasi registre ca si mai sus (R0 si R1) pentru a transmite trei caractere „??”.

Rutina principala (`main`) apeleaza subrutina de initializare (`init`), apoi trece in zona de bucla infinita etichetata `start`. Aceasta zona respecta intocmai pasii algoritmului descris mai sus, iar in final se reapeleaza, creand un ciclul infinit.

Subrutina de transmisie – are rolul de a transmite prin UART1 codul stocat in registrul R0

- scrie valoarea din R0 in registrul SBUF1, comandandu-se astfel transmisia octetului respectiv,
- asteapta setarea flag-ului TI1 care semnalizeaza terminarea transmisiei,
- reseteaza flag-ul TI1 pentru a-l pregati pentru urmatoarea transmisie.

Subrutina de receptie – are rolul de a receptiona un octet prin UART1 si de a-l stoca in acumulator

- asteapta setarea flag-ului RI1 care semnalizeaza faptul ca un octet a fost receptionat,
- stocheaza octetul receptionat (in SBUF1) in registrul acumulator.
- reseteaza flag-ul RI1 pentru a elibera buffer-ul de receptie (incepand de acum se poate receptiona un nou octet de date),

Subrutina de initializare

- dezactiveaza intreruperile
 - pentru detalii vezi explicatiile din Lucrarea nr. 1.
- dezactiveaza watchdog timer-ul
 - pentru detalii vezi explicatiile din Lucrarea nr. 1.
- activeaza crossbar-ul
 - in aceasta aplicatie avem nevoie de activarea UART1, deci vom seta atat bitul ce activeaza crossbar-ul, cat si bitul ce activeaza explicit UART1.
 - pentru detalii despre configurarea crossbar-ului si definitia registrului XBR2 consultati documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 206, pag. 216).
- selecteaza ceasul sistemului si stabileste factorul de divizare a frecventei acestuia
 - in aceasta aplicatie vom folosi oscilatorul intern al SoC-ului cu frecventa de baza 24.5MHz, iar factorul de divizare se seteaza la 1. Motivele pentru care a fost selectata aceasta frecventa vor fi prezentate la punctul urmator.

- pentru detalii despre definitia OSCICN si definitia CLKSEL consultati documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 176, pag. 177).
- configureaza Timer 1
 - conform celor prezentate in capitolul 2.2.1.3. Timer 1 este utilizat pentru generarea baud rate-ului pentru UART 1. Tinand cont de aceasta el trebuie configurat in modul 8-bit auto-reload.
 - in aceasta aplicatie dorim sa setam baud rate-ul la 115200 biti/s. Tinand cont ca am ales deja frecventa ceasului intern (24.5MHz), trebuie alese valorile `T1CLK` si `TH1`, din ecuatia prezentata in capitolul 2.2.1.3., conform tabelului Table 22.1 din documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 286 - 288). Rezulta ca `T1CLK` trebuie sa fie chiar ceasul de sistem, iar valoarea de reload a Timer 1 trebuie sa fie `0x96`.
 - In consecinta codul etichetat `initTimer1` selecteaza modul de functionare al Timer 1 (8-bit auto-reload), seteaza valoarea de reload a `TH1` (`0x96`), selecteaza sursa de sincronizare a Timer 1 (ceasul de sistem) si in final activeaza Timer 1.
- configureaza UART 1
 - UART 1 este setat sa functioneze in modul 8-bit, intrucat nu avem nevoie ca transmisia sa fie extrem de sigura (deci nu vom folosi mecanismul de checksum prezentat in capitolul 2.2.1.2.). Se activeaza de asemenea receptia setand bitul `REN1`.

3 Desfasurarea lucrarii

1. Creati un nou proiect folosind fisierul Z:\Lab8051\Laborator3\serialCommunication.asm urmand pasii prezentati in Lucrarea de laborator nr. 1, la capitolul 2.2.
2. Executati aplicatia.
3. Modificati programul astfel incat dupa transmisia caracterelor corespunzatoare sa se treaca la o linie noua, la capat de rand.
4. Modificati programul astfel incat dupa transmisia caracterelor corespunzatoare sa se treaca la o pagina noua.
5. Modificati programul astfel incat caracterele sa fie transmise la un interval de minim o secunda.
6. Modificati programul pentru receptiona o cifra si a transmite inapoi cifrele i, i+1, ..., 9 (unde i este cifra receptionata).
7. Modificati programul pentru a afisa pe fiecare rand din PuTTY numele si prenumele vostru.
8. Modificati initializarile pentru UART 1 si Timer 1 pentru a face transmisia in modul 9-bit (cu al noualea bit de paritate) la un baud rate de 9600 biti/s.
9. Modificati programul de la punctul 6 astfel incat sa nu se mai faca retransmisia la PuTTY, datele receptionate urmand sa fie afisate pe bara de led-uri astfel:
 - daca se receptioneaza cifra i, i=1..9, sa se aprinda primele i led-uri de la stanga la dreapta.
 - daca se receptioneaza altceva, sa se aprinda led-ul al 10-lea (primul din dreapta).Indicatii:
 - bara de led-uri este conectata la microcontroler conform conform ANEXEI 5.2 din Lucrarea de laborator nr. 2.
10. Modificati programul anterior astfel incat datele receptionate sa fie afisate pe un digit cu 7 segmente astfel:
 - daca se receptioneaza cifra i, i=0..9, sa se aprinda cifra i.
 - daca se receptioneaza altceva, sa se afiseze un cod de eroare (E).Indicatii:
 - afisajul este conectat la microcontroler pe portul 4 (P4) conform ANEXEI 5.2 din Lucrarea de laborator nr. 2.
 - fiecare din subrutinele prezentate in ANEXA 5.3 din Lucrarea de laborator nr. 2 aprind cate o cifra, respectiv simbolul de eroare (E) pe afisajului cu 7 segmente.

4 ANEXE

4.1 Codul aplicatiei SerialCommunication

```

;-----
;
;
; FILE NAME   : serialCommunication.asm
; TARGET MCU  : C8051F040
; DESCRIPTION : UART1 serial communication program.
;
; NOTES:
;
;-----

#include (c8051f040.inc)                ; Include register definition file.

;-----
; RESET and INTERRUPT VECTORS
;-----

                cseg      AT 0x0000      ; Reset Vector
                ljmp      main          ; Locate a jump to the start of code
                                           ;at the reset vector.
;-----
; MAIN PROGRAM CODE SEGMENT
;-----

mainCodeSeg    segment      CODE

                rseg      mainCodeSeg   ; Switch to this code segment.
                using     0             ; Specify register bank for the following
                                           ; program code.

main:          acall      init          ; Initialization of all used SFR's
                mov       SFRPAGE, #UART1_PAGE

start:         call       receiveByte   ; Start the algorithm
                                           ; Receive a character through UART1
                mov       B, A
                clr       C             ; Compare the value in A with #0x41
                subb      A, #0x41      ;and jump to error if lower
                jc        error
                mov       A, #0x5A      ; Compare the value 0x5A with the value
                subb      A, B          ;stored in B and jump to error if lower
                jc        error

                mov       R0, B         ; Store in R0 the previously received ASCII code
                mov       R1, A         ; Init the loop counter
                inc       R1

                loop:      call       sendByte   ; Transmit the character with
                                           ;the ASCII code stored in R0
                inc       R0            ; Increment the ASCII code stored in R0
                djnz      R1, loop      ; Decrement R1 and jump to loop if not zero
                sjmp      start        ; Start over again

                error:      mov       R0, #0x3F   ; Store in R0 the ASCII code of "?"
                mov       R1, #0x03      ; Init the loop counter
                errorLoop: call       sendByte
                djnz      R1, errorLoop ; Decrement R1 and jump if not zero
                sjmp      start        ; Start over again

;-----
; FUNCTION CODE
;-----

init:          clr       EA            ; Disable global interrupts
                mov       WDTCN, #0xDE   ; Disable Watch Dog Timer
                mov       WDTCN, #0xAD
                mov       SFRPAGE, #CONFIG_PAGE ; Use SFRs in the configuration Page
                mov       XBR2, #0x44    ; Enable the crossbar and UART 1

initOscillator: mov       CLKSEL, #0x00
                mov       OSCICN, #0xC3
    
```

```
initTimer1:      mov      SFRPAGE, #TIMER01_PAGE      ; Use SFRs in Timer 1 Page
                  mov      TMOD, #0x20                ; Use Timer 1 in 8-bit auto-reload mode
                  mov      TH1, #0x96                 ; Set the reload value
                  mov      CKCON, #0x10                ; Timer 1 will use the system clock
                  mov      TCON, #0x40                ; Enable Timer 1

initUART1:       mov      SFRPAGE, #UART1_PAGE ; Use SFRs in UART 1 Page
                  mov      SCON1, #0x10             ; Use UART 1 in 8-bit mode

                  ret

sendByte:        mov      SBUF1, R0                  ; Transmit the byte stored in R0
                  jnb      TI1, $                    ; Wait for the end of transmission
                  clr      TI1                      ; Clear the end of transmission flag
                  ret

receiveByte:     jnb      RI1, $                    ; Wait for the end of reception
                  mov      A, SBUF1                  ; Copy the received data in the accumulator
                  clr      RI1                      ; Clear the end of reception flag
                  ret

;-----
; End of file.

END
```

4.2 Tabelul ASCII

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

128	Ç	144	É	161	í	177	␣	193	⌞	209	⌠	225	β	241	±
129	ü	145	æ	162	ó	178	␣	194	⌞	210	⌠	226	Γ	242	≥
130	é	146	Æ	163	û	179		195	⌞	211	⌠	227	π	243	≤
131	â	147	ø	164	ñ	180	⌞	196	—	212	⌞	228	Σ	244	∫
132	ä	148	ö	165	Ñ	181	⌞	197	+	213	⌞	229	σ	245	∫
133	à	149	ò	166	•	182	⌞	198	⌞	214	⌞	230	μ	246	÷
134	â	150	û	167	°	183	⌞	199	⌞	215	⌞	231	τ	247	≈
135	ç	151	ù	168	¿	184	⌞	200	⌞	216	⌞	232	Φ	248	°
136	ê	152	—	169	—	185	⌞	201	⌞	217	⌞	233	⊙	249	.
137	ë	153	Ö	170	¬	186	⌞	202	⌞	218	⌞	234	Ω	250	.
138	è	154	Û	171	½	187	⌞	203	⌞	219	■	235	δ	251	√
139	ì	156	£	172	¾	188	⌞	204	⌞	220	■	236	∞	252	—
140	î	157	¥	173	¡	189	⌞	205	=	221	■	237	φ	253	²
141	ï	158	—	174	«	190	⌞	206	⌞	222	■	238	ε	254	■
142	Ä	159	ƒ	175	»	191	⌞	207	⌞	223	■	239	∩	255	
143	Å	160	á	176	␣	192	⌞	208	⌞	224	α	240	≡		